

Arquitetura RISC-V

- Arquitetura de 32 bits
- 32 registradores de propósito geral
- 32 registradores para float

Estrutura de código

- `.data` - Início da seção de dados estáticos
- `.text` - Início da seção de código
- `.global label` - Entry point. Permite que outros arquivos chamem essa função durante o processo de link.

Diretivas

- `.align n` - Alinha a memória em 2^n bytes. Força o próximo dado/instrução a começar em um endereço de memória alinhado, utilizando padding.
 - `n = 0` para strings.
 - `n = 2` para instruções.
- `label: .asciz "str"` - Guarda string e encerra ela com `\0`
- `label: .byte b1, ..., bn` - Reserva e inicializa cada byte sequencialmente.
- `label: .word w1, ..., wn` - Reserva e inicializa cada word sequencialmente.
- `label: .double d1, ..., dn` - Reserva e inicializa cada double sequencialmente.
- `label: .space n` - Reserva n bytes

Registradores

- `ra` - Return address
- `sp` - Stack pointer
- `t0-t6` - Temporary
- `a0-a6` - Function arguments
- `a7` - Syscall register
- `s0-s11` - Saved values

Comandos

[Risc-V Cheat Sheet](#)

Aritmética

- `add rd, rs1, rs2` - `rd = rs1 + rs2` (2 registradores)
- `addi rd, rs1, imm` - `rd = rs1 + imm` (1 registrador, 1 valor imediato/constante)
- `sub rd, rs1, rs2` - `rd = rs1 - rs2`
- `mul rd, rs1, rs2` - `rd = rs1 * rs2`
- `div rd, rs1, rs2` - `rd = rs1 / rs2`
- `rem rd, rs1, rs2` - `rd = rs1 % rs2`
- `remu rd, rs1, rs2` - `rd = rs1 % rs2` (unsigned remainder)
- `xor rd, rs1, rs2` - `rd = rs1 xor rs2`

Load/Store

- `la rd, label` - `rd = label_addr`
- `lw rd, imm(rs1)` - `rd = mem[rs1+imm]` (word = 4 bytes)
- `lb rd, imm(rs1)` - `rd = mem[rs1+imm]` (byte)
- `sw rs2, imm(rs1)` - `mem[rs1+imm] = rs2`

- `sb rs2, imm(rs1) - mem[rs1+imm] = rs2`

Jump/Function

- `j label - pc += label` (Obs: Usa half-words para pular: `jmp 4 - pc += 4 * 2 bytes`)
- `jal rd, label - rd = pc+4; pc += label` (salva endereço de retorno)
- `jr ra - pc = ra`
- `call symbol - ra = pc+4; pc = &symbol`
- `ret - pc = ra`

Branch (imm pode também ser um label)

- `beq rs1, rs2, imm - if(rs1 == rs2) pc += imm`
- `bne rs1, rs2, imm - if(rs1 != rs2) pc += imm`
- `blt rs1, rs2, imm - if(rs1 < rs2) pc += imm`
- `ble rs1, rs2, imm - if(rs1 <= rs2) pc += imm`
- `bgt rs1, rs2, imm - if(rs1 > rs2) pc += imm`
- `bge rs1, rs2, imm - if(rs1 >= rs2) pc += imm`
- `bnez rs1, imm - if(rs1 != 0) pc += imm`

Comparadores

- `slt rd, rs1, rs2 - rd = (rs1 < rs2)`
- `slti rd, rs1, imm - rd = (rs1 < imm)`

Ecalls

- `a7` - Armazena número da ecall
- `ecall` - Chama a ecall

Chamadas ao sistema (ecall)

Name	Number	Description	Inputs	Ouputs
PrintInt	1	Prints an integer	a0 = integer to print	N/A
PrintFloat	2	Prints an floating point number	fa0 = float to print	N/A
PrintDouble	3	Prints a double precision floating point number	fa0 = double to print	N/A
PrintString	4	Prints a null-terminated string to the console	a0 = the address of the string	N/A
ReadInt	5	Read an int from input console	N/A	a0 = the int
ReadFloat	6	Read a float from input console	N/A	fa0 = the float
ReadDouble	7	Reads a double from input console	N/A	fa0 = the double
ReadString	8	Reads a string from the console	a0 = address of input buffer a1 = maximum number of characters to read	N/A
Sbrk	9	Allocate heap memory	a0 = amount of memory in bytes	a0 = address to the allocated block
Exit	10	Exits the program with code 0	N/A	N/A
PrintChar	11	Prints an ascii character	a0 = character to print (only lowest byte is considered)	N/A
ReadChar	12	Read a character from input console	N/A	a0 = the character
Exit2	93	Exits the program with a code	a0 = the number to exit with	N/A

Alocando na stack

Empilhando

- `addi sp, sp, -8` - Reserva 8 words
- `sw a0, 4(sp)` - Salva `a0` na stack (sp+4)
- `sw ra, 0(sp)` - Salva `ra` na stack (sp)

Desempilhando

- `lw a0, 4(sp)`
- `lw ra, 0(sp)`
- `addi sp, sp, 8`

Tipos das instruções

31	30	25	24	21	20	19	15	14	12	11	8	7	6	0			
funct7				rs2			rs1		funct3		rd			opcode		R-type	
imm[11:0]							rs1		funct3		rd			opcode		I-type	
imm[11:5]				rs2			rs1		funct3		imm[4:0]			opcode		S-type	
imm[12]		imm[10:5]			rs2			rs1		funct3		imm[4:1]		imm[11]		opcode	B-type
imm[31:12]										rd			opcode		U-type		
imm[20]		imm[10:1]			imm[11]		imm[19:12]			rd			opcode		J-type		

Tipo R (register)

Operações matemáticas e lógicas: `add` , `sub` , `and` , `or` , `sll`

- `opcode` - Código da operação
- `rd` - Endereço do registrador destino
- `funct3` - Auxílio para definição da operação
- `rs1` - Endereço do primeiro registrador de origem
- `rs2` - Endereço do segundo registrador de origem
- `funct7` - Auxílio para definição da operação

Ex: `add rd, rs1, rs2`

```
00000000 01000 01001 000 10010 0110011
```

Tipo I (immediate)

Operações aritméticas com constantes ou instruções de leitura da memória: `addi` , `lw` , `andi` , `jalr`

- `opcode` - Código da operação
- `rd` - Endereço do registrador destino
- `funct3` - Auxílio para definição da operação
- `rs1` - Endereço do primeiro registrador de origem
- `imm` - Valor imediato

Ex: `lw s2, 0(sp)`

```
00000000000000 00010 010 10010 0000011
```

Tipo S (Store)

Escrever dados de um registrador para a memória: `sw`

- `opcode` - Código da operação
- `imm[4:0]` e `imm[11:5]` - Valor imediato (Separa os bits)
- `funct3` - Auxílio para definição da operação
- `rs1` - Endereço do primeiro registrador de origem
- `rs2` - Endereço do segundo registrador de origem

Ex: `sw s2, 0(sp)`

```
00000000 10010 00010 010 00000 0100011
```

Tipo B (branch)

Desvios condicionais: `beq` , `bne` , etc

- `opcode` - Código da operação
- `imm[4:0]` , `imm[11]` , `imm[11:5]` , `imm[12]` - Valor imediato
- `funct3` - Auxílio para definição da operação
- `rs1` - Endereço do primeiro registrador de origem
- `rs2` - Endereço do segundo registrador de origem

Ex: `beq s1, s0, 4`

```
0 000000 01000 01001 000 0100 0 1100011
```

Tipo U (Upper immediate)

Carregar números muito grandes diretamente nos bits superiores de um registrador: `lui` , `auipc`

- `opcode` - Código da operação
- `rd` - Endereço do registrador destino
- `imm[31:12]` - Valor imediato

Ex: `lui s0, 0x01234`

```
[0000 0001 0010 0011 0100] 01000 0110111
```

Tipo J

Pulos incondicionais de longo alcance (chamadas de função e desvios obrigatórios): `jal`

- `opcode` - Código da operação
- `rd` - Endereço do registrador destino
- `imm[20]` , `imm[10:1]` , `imm[11]` , `imm[19:12]` - Valor imediato

Ex: `jal s0, -4`

```
1 1111111100 1 11111111 01000 1101111
```